

Individual Project – Data Structures

Spring 2024

Deliveries

- This assignment requires one .cpp file submission.
- Write necessary Comments in a way that it should be clear and concise, providing meaningful explanations of the code's purpose or functionality.
- Do not upload a code snippet, the only files you submit for the code should be the source files.
- Do not submit any zip or compressed files, binary files, or any other generated files.
- Do not put the code inside the PDF file.
- Submission of unnecessary files or binary files in addition to source files will make you lose the points of the assignment.
- We evaluate your code using the following C++ online compiler. You will not get points if your assignment does not compile with this. https://www.onlinegdb.com/online_c++_compiler

Learning Outcomes

Upon completing this programming assignment, you will be able to:

1. Implementing a stack-based calculator provides hands-on experience with fundamental data structures, particularly stacks. This includes understanding how stacks work, their basic operations (push, pop, peek), and their application in solving problems.
2. Developing the calculator involves devising algorithms for converting infix expressions to postfix notation and evaluating postfix expressions. This fosters algorithmic thinking skills, helping learners break down complex problems into manageable steps.
3. Dealing with error conditions such as stack overflow, stack underflow, invalid input expressions, and provides practical experience in error handling and robust programming practices.

Project Title:

Calculator Application with Postfix Expression Evaluation

Project Description:

The Calculator Application with Postfix Expression Evaluation aims to develop a calculator tool that supports arithmetic operations using infix notation. This project focuses on utilizing stacks to parse and evaluate postfix expressions, providing users with a flexible and efficient calculation tool. The application will follow defined rules for operand and operator handling, accommodating multi-digit input and considering rounding where necessary.

Tasks:

1. The calculator is equipped to handle various arithmetic operations, including **addition, subtraction, multiplication, division, modulo, and exponentiation**. For example, users can input expressions in infix format such as $(2 + 3) - 2$, and the calculator will first display the equivalent postfix format, $23+2-$, before providing the final calculation result, which in this case is **3**. (30 points)

2. **Only integer operands are allowed, and the calculator does not accept input containing alphabetic characters or special characters other than mathematical operators.** The postfix expression evaluation should yield results in floating-point format, rounded to **three decimal points**. If the input expression contains alphabetic letters or special characters (not arithmetic operators e.g., "+", "-", "*", "/", "%", and "^"), the program should send an error message indicating the character that caused the error. **(15 points)**
3. It is capable of handling **imbalanced parentheses or an invalid number of operators**. For example, if the input is **1+**, the program should detect the imbalance or invalid number of operands/operators and send an appropriate error message. **(20 points)**
4. The code can handle all types of parentheses, including round brackets, curly brackets, and square brackets. Whether the input is **{2 + 3* (2 +2)} - 4** or **(2+3 *(2+2))-4**, the program can recognize both infix notations as correct and provide the accurate result **(10 points)**
5. For correctly running all the test cases provided in the attached PDF (StackbasedCal_test_cases.pdf), you will receive **25 points**.
6. **(Bonus):** Your calculator accepts operands with multi-digit non-negative numbers. For example, the infix expression **28 - 3** will return a value of **25**. **(20 points)**

Input format:

The input consists of a single string, denoted as S, which may contain alphabetic characters, digits, parentheses, numbers, arithmetic operators, and other characters.

Output format:

1. If your infix notation is not correct, the program will send an error message with invalid character.
2. If the input infix notation is correct:
 - a. The program first displays the postfix notation of the given infix expression.
 - b. Then, it shows the result of the postfix evaluation in floating-point format, rounded up to three decimal points.

Implementation Details

The implementation of the stack-based calculator must be done using **arrays**, and you cannot utilize any STL library, including **<stack>**, for implementing stack operations. You must implement the project individually. **It is not a team project.**

Hints

You may utilize the **stoi()** function, which is a standard library function converting a string into an integer. Additionally, the **fmod()** function is available for modulo operations. For setting precision points, **<iomanip>** can be utilized. To tackle the bonus part, consider treating the postfix expression not as a string but as an array of strings.

Examples of inputs and expected outputs.

Sample 1.

Input:

```
2+3-5+2^2
```

Output:

Postfix expression: 23+5-22^+

Result: 4.000

Sample 2.

Input:

2=3

Output:

Error: Invalid character '=' in expression

Sample 3.

Input:

(2+3*(2+2))-4

Output:

Postfix expression: 2322+*+4-

Result: 10.000

Sample 4.

Input:

{2+3*(2+2))-4

Output:

Postfix expression: 2322+*+4-

Result: 10.000

Sample 5.

Input:

(7+3)*(6-2)/2

Output:

Postfix expression: 73+62-*2/

Result: 20.000

Sample 6.

Input:

$(8-3)/(4+2)$

Output:

Postfix expression: 83-42+/
Result: 0.833

Bonus problem Testing

Sample 7.

Input:

$(28-3)/(4+2)$

Output:

Postfix expression: 283-42+/
Result: 4.167

Appendix

Test case samples are also provided in a PDF file. The bellow is the main function of the program looks like:

```
int main() {
    std::string infixExpression;
    std::cout << "Enter an infix expression: "; // Prompt the user to enter an input in infix format
    getline(std::cin, infixExpression);

    String postfixExp = to_postfix(infixExpression); // Transform the infix expression to postfix

    print_postfix(postfixExp); // Print the postfix expression

    double result = evaluatePostfix(postfixExp); // Evaluate the postfix expression

    return 0;
}
```